

FECAP – Fundação Escola de Comércio Álvares Penteado

Curso de Ciência da Computação

SISTEMA IOT PARA MONITORAMENTO CLIMÁTICO PREVENTIVO EM ACERVOS DE MUSEUS

Arkanis

Entrega 02 – Implementação e Validação do Sistema

Luciano Reis Massaro – 23025304

Aleff Silva Souza – 23025514

Matheus Moraes Zimmer – 23025264

João Paulo Souza Colombo – 23025479

São Paulo

2026

1. Objetivo do sistema

O projeto Arkanis tem como objetivo desenvolver uma plataforma IoT inteligente capaz de monitorar continuamente as condições ambientais em ambientes expositivos e reservas técnicas de museus.

A solução permite acompanhar variáveis como temperatura, umidade, pressão, presença de gás e detecção de fogo, auxiliando na identificação de condições inadequadas para preservação de acervos culturais.

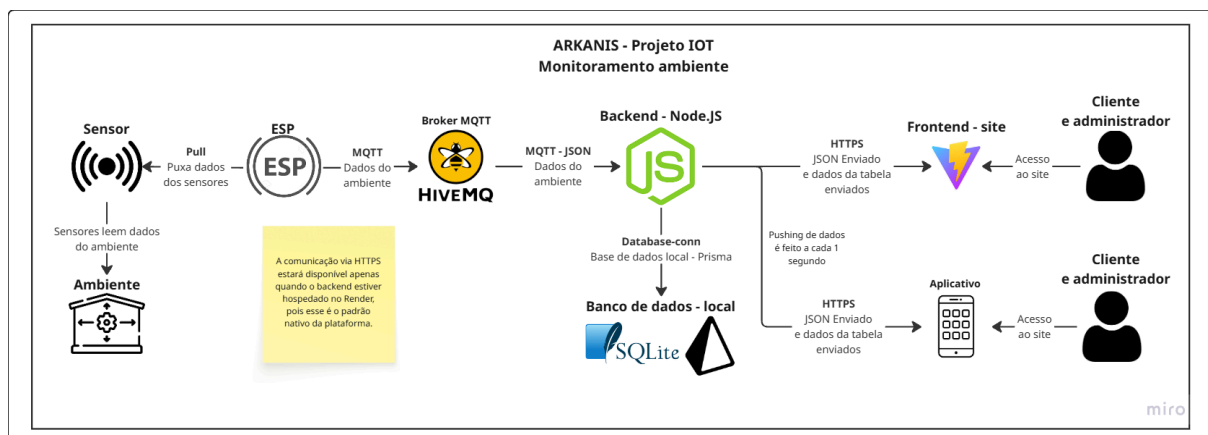
O sistema busca apoiar ações preventivas e preditivas de conservação, oferecendo:

- Monitoramento em tempo real;
- Detecção de anomalias ambientais;
- Alertas visuais e sonoros;
- Acionamento de atuadores;
- Histórico de leituras;
- Dashboard web;
- Aplicativo mobile;
- Relatórios consolidados.

2. Descrição da arquitetura final implementada

2.1 Visão geral da arquitetura

A arquitetura final implementada segue o fluxo:



A arquitetura foi organizada para que o ESP32 seja responsável pela coleta das leituras e pelo acionamento físico dos atuadores. O backend centraliza o recebimento, validação, processamento e armazenamento dos dados. O site e o aplicativo consomem os dados por API REST e recebem atualizações em tempo real via Socket.IO.

2.2 Componentes da arquitetura

2.2.1 Sensores

O sistema utiliza sensores conectados ao ESP32 para capturar informações ambientais relevantes para a conservação de acervos.

Sensores utilizados:

- **BME280**: temperatura, umidade e pressão;
- **MQ-2**: detecção de gás/fumaça;
- **Sensor de fogo**: detecção de chama/incêndio.

As leituras são feitas continuamente no loop principal do ESP32.

2.2.2 Módulo de processamento — ESP32

O ESP32 foi programado em **C++** e executa as principais tarefas embarcadas do sistema:

- Conexão com a rede Wi-Fi;
- Conexão segura com o broker MQTT HiveMQ Cloud;
- Leitura dos sensores;
- Processamento dos estados do ambiente;
- Acionamento de LEDs, buzzer e relé;
- Publicação do payload MQTT.

O código implementa uma máquina de estados com três estados principais:

NORMAL
ALERTA
EMERGENCIA

A decisão do estado ocorre com base nos limites definidos:

Temperatura limite: 30 °C

Umidade limite: 60 %

Gás limite: 550

O estado muda para **EMERGENCIA** quando há detecção de fogo ou gás acima do limite.

Muda para **ALERTA** quando temperatura ou umidade ultrapassam os limites definidos.

Caso contrário, permanece como **NORMAL**.

2.2.3 Atuadores

O sistema utiliza atuadores para responder fisicamente às condições detectadas.

Atuadores implementados:

- **LED verde:** estado normal;
- **LED amarelo:** estado de alerta;
- **LED vermelho:** estado de emergência;
- **Buzzer:** acionado em emergência;
- **Relé da ventoinha:** acionado em alerta.

A lógica dos atuadores segue a máquina de estados:

Estado	LED Verde	LED Amarelo	LED Vermelho	Buzzer	Ventoinha
NORMAL	Ligado	Desligado	Desligado	Desligado	Desligada
ALERTA	Desligado	Ligado	Desligado	Desligado	Ligada
EMERGENCIA	Desligado	Desligado	Ligado	Ligado	Desligada

No estado **ALERTA**, a ventoinha é ligada para atuar preventivamente na condição ambiental. No estado **EMERGENCIA**, o LED vermelho e o buzzer são acionados para indicar risco crítico.

2.2.4 Comunicação MQTT

A comunicação entre o ESP32 e o backend ocorre por MQTT, utilizando o broker HiveMQ Cloud.

O ESP32 publica as leituras no tópico:

flex/sala_museu_1/sensor_01

Payload publicado:

```
{
  "device_id": "sensor_01",
  "environment_id": "sala_museu_1",
  "temperature": 25.4,
  "humidity": 60.1,
  "pressure": 1012.5,
  "gas": 320,
  "fogo": false,
  "estado": "NORMAL"
}
```

O backend assina o tópico configurado, recebe o payload, identifica o dispositivo, associa a leitura ao ambiente correto e salva a informação no banco de dados.

2.2.5 Backend

O backend foi desenvolvido em Node.js com Express.

Principais responsabilidades:

- Conectar ao broker MQTT;
- Receber leituras do ESP32;
- Validar os dados recebidos;
- Identificar dispositivo e ambiente;
- Persistir as leituras no banco;
- Registrar logs;
- Gerar alertas;
- Fornecer API REST;
- Enviar atualizações em tempo real via Socket.IO;
- Gerenciar ambientes, dispositivos, usuários, histórico e relatórios.

O backend foi publicado no Render e está disponível em:

<https://projeto1-4rsx.onrender.com>

Endpoint de verificação:

<https://projeto1-4rsx.onrender.com/healthz>

2.2.6 Banco de dados

Foi utilizado SQLite com Prisma ORM.

O banco armazena informações como:

- Usuários;
- Ambientes;
- Dispositivos;
- Leituras;
- Alertas;
- Logs;
- Configurações MQTT.

A escolha do SQLite foi feita para manter o projeto simples, leve e adequado ao contexto acadêmico.

2.2.7 Site web

O site web foi desenvolvido com React e Vite.

Funcionalidades implementadas:

- Login;
- Dashboard em tempo real;
- Visualização de leituras ambientais;
- Visualização do JSON recebido;
- Gráfico de leituras;
- CRUD de ambientes;
- CRUD de dispositivos;
- Configuração MQTT;
- Histórico;
- Relatórios CSV;
- Logs;
- Gestão de usuários.

2.2.8 Aplicativo mobile

O aplicativo mobile foi desenvolvido com React Native e Expo.

Funcionalidades implementadas:

- Login;
- Aceite LGPD;
- Lista de ambientes;
- Detalhes do ambiente;
- Últimas leituras;
- Gráfico por período;
- Central de alertas;
- Preferências;
- Comunicação com backend via API REST;
- Atualização em tempo real via Socket.IO.

3. Código-fonte organizado e versionado

O código-fonte foi organizado no repositório GitHub do projeto.

Estrutura principal da Entrega 2:

```
src/Entrega2/SiteApp/  
├── flex-iot-dashboard-refatorado/  
│   ├── backend/  
│   └── frontend/  
└── flex-iot-mobile-expo/
```

3.1 Backend

Caminho:

src/Entrega2/SiteApp/flex-iot-dashboard-refatorado/backend

Contém:

- Rotas da API;
- Serviços MQTT;
- Serviço de monitoramento offline;
- Middleware de autenticação;
- Prisma schema;
- Seed inicial;
- Configuração do servidor.

3.2 Frontend web

Caminho:

src/Entrega2/SiteApp/flex-iot-dashboard-refatorado/frontend

Contém:

- Interface web;
- Dashboard;
- Telas de CRUD;
- Histórico;
- Relatórios;
- Configuração MQTT.

3.3 Aplicativo mobile

Caminho:

src/Entrega2/SiteApp/flex-iot-mobile-expo

Contém:

- Telas do app;
- Serviços de API;
- Serviço de Socket.IO;
- Contexto de autenticação;
- Preferências;
- Componentes reutilizáveis.

4. Evidência da integração entre sensores, processamento e atuadores

Gravamos um vídeo para evidenciar o funcionamento do sistema, sensores, processamento e atuadores, este vídeo se encontra na mesma pasta do github que este arquivo.

A integração ocorre da seguinte forma:

1. O sensor BME280 coleta temperatura, umidade e pressão.
2. O sensor MQ-2 coleta nível de gás/fumaça.
3. O sensor de fogo detecta presença de chama.
4. O ESP32 processa as leituras.
5. A máquina de estados define se o ambiente está NORMAL, em ALERTA ou em EMERGENCIA.
6. Os atuadores são acionados conforme o estado.
7. O ESP32 publica o JSON no tópico MQTT.
8. O backend recebe os dados.
9. O backend salva a leitura no banco.
10. O site e o app são atualizados em tempo real.

4.1 Estado NORMAL

Condição:

Temperatura abaixo de 30 °C

Umidade abaixo de 60 %

Gás abaixo de 550

Fogo não detectado

Ações:

- LED verde ligado;
- LED amarelo desligado;
- LED vermelho desligado;
- Buzzer desligado;
- Ventoinha desligada;
- Publicação MQTT com estado NORMAL.

4.2 Estado ALERTA

Condição:

Temperatura ≥ 30 °C

ou

Umidade ≥ 60 %

Ações:

- LED verde desligado;
- LED amarelo ligado;
- LED vermelho desligado;
- Buzzer desligado;
- Ventoinha ligada;
- Publicação MQTT com estado ALERTA.

4.3 Estado EMERGENCIA

Condição:

Fogo detectado

ou

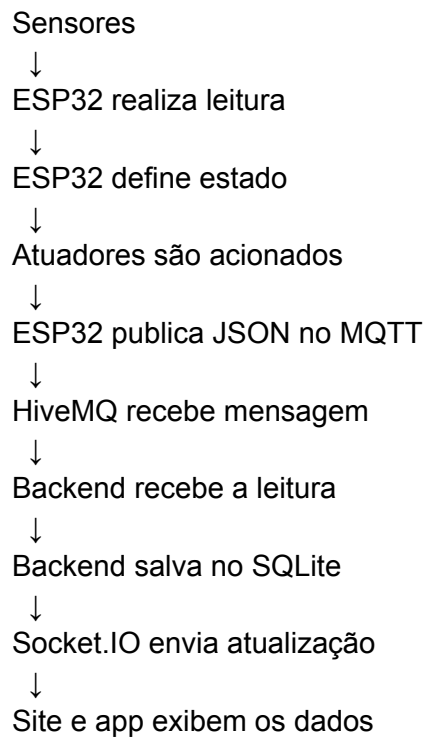
Gás ≥ 550

Ações:

- LED verde desligado;
- LED amarelo desligado;
- LED vermelho ligado;
- Buzzer ligado;
- Ventoinha desligada;
- Publicação MQTT com estado EMERGENCIA.

5. Demonstração do fluxo completo de dados e ações

5.1 Fluxo com ESP32



5.2 Fluxo sem ESP físico

Quando o ESP32 não está disponível, o sistema pode ser testado pelo Postman, simulando a leitura que chegaria pelo MQTT.

Endpoint:

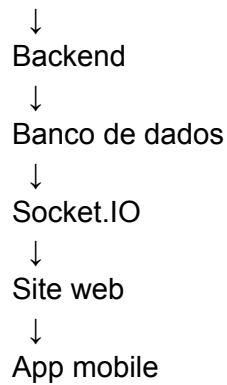
POST <https://projeto1-4rsx.onrender.com/api/test-reading>

Body:

```
{
  "topic": "flex/sala_museu_1/sensor_01",
  "temperature": 28.5,
  "humidity": 52.1,
  "pressure": 1012.8,
  "gas": 410,
  "fogo": false,
  "estado": "NORMAL"
}
```

Esse teste percorre o fluxo:

Postman



6. Registro dos testes realizados

ID	Teste realizado	Resultado esperado	Status
T01	Conectar ESP32 ao Wi-Fi	ESP32 conectado à rede	Concluído
T02	Conectar ESP32 ao HiveMQ	Cliente MQTT conectado	Concluído
T03	Ler temperatura, umidade e pressão	Valores exibidos no Serial Monitor	Concluído
T04	Ler sensor MQ-2	Valor analógico exibido	Concluído
T05	Ler sensor de fogo	Estado de fogo detectado ou não	Concluído
T06	Estado NORMAL	LED verde ligado	Concluído
T07	Estado ALERTA	LED amarelo e ventoinha ligados	Concluído
T08	Estado EMERGENCIA	LED vermelho e buzzer ligados	Concluído
T09	Publicar payload MQTT	Mensagem enviada ao tópico correto	Concluído
T10	Backend receber leitura	Leitura salva no banco	Concluído
T11	Dashboard web atualizar	Cards e gráfico atualizados	Concluído

T12	App mobile atualizar	Leituras aparecem no app	Concluído
T13	Relatório CSV	Arquivo gerado com leituras	Concluído
T14	Teste via Postman	Simulação salva e exibida	Concluído
T15	Login web e mobile	Usuário autenticado	Concluído

7. Validação dos requisitos

7.1 Requisitos funcionais

Requisito	Implementação	Status
Monitorar temperatura	BME280 + dashboard/app	Concluído
Monitorar umidade	BME280 + dashboard/app	Concluído
Detectar anomalias	Máquina de estados no ESP32	Concluído
Comunicação IoT	MQTT com HiveMQ	Concluído
Processamento	ESP32 + backend	Concluído
Atuadores	LEDs, buzzer e ventoinha	Concluído
Dashboard	Site React/Vite	Concluído
Aplicativo mobile	React Native/Expo	Concluído
Histórico de leituras	Backend + banco SQLite	Concluído
Relatórios	CSV	Concluído
Logs	Backend registra eventos	Concluído
Alertas	Estado ALERTA/EMERGENCIA	Concluído

--	--	--

7.2 Requisitos não funcionais

Requisito	Implementação	Status
Segurança	JWT e senha hash	Concluído
Comunicação externa	Backend HTTPS no Render	Concluído
Usabilidade	Interfaces web e mobile simples	Concluído
Manutenibilidade	Código separado em backend, frontend e mobile	Concluído
Desempenho	Limitação/amostragem de dados nos gráficos	Concluído
Confiabilidade	Reconexão MQTT no ESP32	Concluído
Persistência	SQLite com Prisma	Concluído
Portabilidade	App Expo e backend Node.js	Concluído

8. Backlog final atualizado - Projects do github atualizado

8.1 Tarefas concluídas

ID	Tarefa	Status
B01	Definir objetivo e arquitetura do sistema	Concluído
B02	Implementar leitura dos sensores no ESP32	Concluído

B03	Implementar conexão Wi-Fi	Concluído
B04	Implementar conexão MQTT com HiveMQ	Concluído
B05	Criar máquina de estados NORMAL/ALERTA/EMERGENCIA	Concluído
B06	Acionar LEDs conforme estado	Concluído
B07	Acionar buzzer em emergência	Concluído
B08	Acionar ventoinha em alerta	Concluído
B09	Publicar payload JSON via MQTT	Concluído
B10	Criar backend Node.js/Express	Concluído
B11	Integrar backend com MQTT	Concluído
B12	Criar banco SQLite com Prisma	Concluído
B13	Criar autenticação JWT	Concluído
B14	Criar site web	Concluído
B15	Criar dashboard em tempo real	Concluído
B16	Criar histórico de leituras	Concluído
B17	Criar relatório CSV	Concluído
B18	Criar aplicativo mobile	Concluído
B19	Integrar app com API	Concluído

B20	Publicar backend no Render	Concluído
B21	Gerar APK do aplicativo	Concluído
B22	Documentar projeto	Concluído

8.2 Pendências e melhorias futuras

ID	Tarefa	Prioridade
P01	Implementar recuperação real de senha por e-mail	Média
P02	Implementar notificações push reais	Média
P03	Criar exportação PDF	Baixa
P04	Implementar cálculo de disponibilidade (uptime/downtime) por dispositivo	Média
P05	Implementar reconhecimento manual de alertas	Média
P06	Migrar SQLite para PostgreSQL em produção	Alta
P07	Criar regras de alerta por janela móvel	Média
P08	Melhorar controle visual por perfil	Média
P09	Criar testes automatizados de API	Baixa

9. Análise crítica das limitações do sistema

9.1 SQLite no Render

O sistema utiliza SQLite por simplicidade acadêmica. Porém, no Render, o armazenamento local pode ser reinicializado em caso de redeploy ou reinício da instância.

Melhoria proposta: migrar para PostgreSQL gerenciado em ambiente produtivo.

9.2 Recuperação de senha simulada

A recuperação de senha foi implementada de forma simplificada no aplicativo.

Melhoria proposta: integrar serviço de e-mail com token temporário para redefinição de senha.

9.3 Notificações push

O aplicativo apresenta alertas visuais, mas ainda não implementa notificações push reais.

Melhoria proposta: integrar Expo Notifications ou Firebase Cloud Messaging.

9.4 Uptime e disponibilidade

O sistema identifica status online/offline, mas ainda não calcula uptime detalhado por período.

Melhoria proposta: implementar cálculo de disponibilidade por dispositivo com base em eventos e última leitura recebida.

9.5 Exportação PDF

O sistema gera relatórios CSV, mas não exporta PDF.

Melhoria proposta: adicionar relatório PDF com resumo estatístico e histórico.

9.6 Deduplicação de mensagens

O payload atual não possui identificador único da mensagem.

Melhoria proposta: adicionar messageId ou número sequencial ao payload para evitar duplicidade.

10. Conclusão

A implementação final do sistema Arkanis demonstrou a integração entre sensores, módulo de processamento, atuadores e comunicação.

O ESP32 realiza a leitura dos sensores, processa os estados ambientais, aciona LEDs, buzzer e ventoinha, e publica as informações via MQTT. O backend recebe os dados, salva no banco, gera alertas e envia atualizações em tempo real para o site e aplicativo.

A solução implementada valida técnica e funcionalmente a proposta definida na Entrega 1, demonstrando um sistema IoT funcional para monitoramento ambiental em museus e apoio à conservação preventiva de acervos culturais.

Apesar de existirem melhorias futuras, o projeto atende ao fluxo principal proposto e apresenta uma base sólida para evolução em um cenário real de conservação ambiental inteligente.